

Compact Schema 📌

(schema in detail after this)

NAWI Test Report — Compact Database Schema

Total tables: 21

1. laboratories

Stores registered testing laboratories.

Variable	Type
laboratory_id	UUID
laboratory_code	VARCHAR
name	VARCHAR
registration_number	VARCHAR
address_line_1	VARCHAR
address_line_2	VARCHAR
city	VARCHAR
state	VARCHAR
pincode	VARCHAR
country	VARCHAR
phone	VARCHAR
email	VARCHAR
website	VARCHAR
accreditation_body	VARCHAR
accreditation_number	VARCHAR
accreditation_valid_from	DATE

Variable	Type
accreditation_valid_until	DATE
status	ENUM
created_at	TIMESTAMP
updated_at	TIMESTAMP

2. users

Stores all system users and their roles.

Variable	Type
user_id	UUID
laboratory_id	UUID FK
employee_id	VARCHAR
first_name	VARCHAR
last_name	VARCHAR
email	VARCHAR
phone	VARCHAR
password_hash	VARCHAR
role	ENUM
designation	VARCHAR
qualification	TEXT
authorization_scope	JSON
status	ENUM
last_login_at	TIMESTAMP
created_by	UUID FK
created_at	TIMESTAMP

Variable	Type
updated_at	TIMESTAMP

3. instruments

Stores instruments submitted/registered for testing.

Variable	Type
instrument_id	UUID
laboratory_id	UUID FK
instrument_code	VARCHAR
manufacturer	VARCHAR
model	VARCHAR
type_designation	VARCHAR
serial_number	VARCHAR
instrument_type	ENUM/VARCHAR
instrument_category	VARCHAR
Max	DECIMAL
Min	DECIMAL
e	DECIMAL
d	DECIMAL
n	INTEGER
unit	VARCHAR
tare_type	VARCHAR
zero_setting_type	VARCHAR
indication_type	VARCHAR
load_cell_manufacturer	VARCHAR
load_cell_model	VARCHAR

Variable	Type
load_cell_capacity	DECIMAL
software_version	VARCHAR
firmware_version	VARCHAR
power_supply	VARCHAR
interfaces	JSON
operating_temp_min	DECIMAL
operating_temp_max	DECIMAL
status	ENUM
created_by	UUID FK
created_at	TIMESTAMP
updated_at	TIMESTAMP

4. standards

Stores standards and their versions.

Variable	Type
standard_id	UUID
standard_code	VARCHAR
title	VARCHAR
version	VARCHAR
edition_year	INTEGER
effective_from	DATE
effective_until	DATE
source_document	VARCHAR
source_url	TEXT
status	ENUM

Variable	Type
created_at	TIMESTAMP

5. test_definitions

Defines available tests, their procedures, inputs, and acceptance rules.

Variable	Type
test_definition_id	UUID
standard_id	UUID FK
test_code	VARCHAR
test_name	VARCHAR
test_category	VARCHAR
description	TEXT
reference_clause	VARCHAR
procedure	TEXT/JSON
required_inputs	JSON
calculation_type	VARCHAR
acceptance_rule	JSON
required_equipment	JSON
is_mandatory	BOOLEAN
active	BOOLEAN

6. test_applicability_rules

Defines when a test applies or is N/A for an instrument.

Variable	Type
applicability_rule_id	UUID

Variable	Type
test_definition_id	UUID FK
instrument_condition	JSON
applicable	BOOLEAN
reason	TEXT
priority	INTEGER
standard_clause	VARCHAR

7. mpe_rules

Stores permissible-error rules used by calculations.

Variable	Type
mpe_rule_id	UUID
standard_id	UUID FK
accuracy_class	VARCHAR
range_min_e	DECIMAL
range_max_e	DECIMAL
mpe_value_e	DECIMAL
mpe_unit_type	VARCHAR
condition	VARCHAR
test_type	VARCHAR
reference_clause	VARCHAR

8. test_sessions

Represents an actual testing event.

Variable	Type
test_session_id	UUID
laboratory_id	UUID FK
instrument_id	UUID FK
tester_id	UUID FK
reviewer_id	UUID FK
standard_id	UUID FK
session_number	VARCHAR
application_number	VARCHAR
test_type	VARCHAR
started_at	TIMESTAMP
completed_at	TIMESTAMP
status	ENUM
overall_result	ENUM
remarks	TEXT
created_at	TIMESTAMP
updated_at	TIMESTAMP

9. test_session_tests

Connects a test session with its applicable/N/A tests.

Variable	Type
session_test_id	UUID
test_session_id	UUID FK
test_definition_id	UUID FK
applicability_status	ENUM
na_reason	TEXT

Variable	Type
status	ENUM
result	ENUM
started_at	TIMESTAMP
completed_at	TIMESTAMP

10. test_observations

Stores raw values entered/imported during testing.

Variable	Type
observation_id	UUID
session_test_id	UUID FK
parameter_name	VARCHAR
parameter_code	VARCHAR
value_numeric	DECIMAL
value_text	TEXT
unit	VARCHAR
timestamp	TIMESTAMP
sequence_no	INTEGER
source	ENUM
entered_by	UUID FK
remarks	TEXT

11. observation_imports

Tracks Excel/CSV observation imports. (this is different may not be used)

Variable	Type
import_id	UUID
test_session_id	UUID
uploaded_by	UUID
file_name	VARCHAR
file_type	VARCHAR
file_path	TEXT
row_count	INTEGER
valid_rows	INTEGER
invalid_rows	INTEGER
validation_status	ENUM
validation_errors	JSON
uploaded_at	TIMESTAMP

12. test_calculations

Stores values calculated by the system.

Variable	Type
calculation_id	UUID
session_test_id	UUID
observation_id	UUID
calculation_type	VARCHAR
input_values	JSON
formula	TEXT
calculated_value	DECIMAL
unit	VARCHAR
calculation_version	VARCHAR

Variable	Type
calculated_at	TIMESTAMP

13. test_results

Stores final interpreted test results.

Variable	Type
result_id	UUID
session_test_id	UUID
measured_value	DECIMAL
mpe_value	DECIMAL
error_value	DECIMAL
corrected_error	DECIMAL
acceptance_condition	TEXT
pass_fail	ENUM
result_summary	TEXT
calculation_version	VARCHAR
evaluated_at	TIMESTAMP

14. environmental_conditions

Stores environmental conditions during testing.

Variable	Type
environment_id	UUID
test_session_id	UUID
temperature	DECIMAL
temperature_unit	VARCHAR

Variable	Type
humidity	DECIMAL
humidity_unit	VARCHAR
pressure	DECIMAL
pressure_unit	VARCHAR
recorded_at	TIMESTAMP
recorded_by	UUID
source	VARCHAR
remarks	TEXT

15. test_equipment

Stores laboratory equipment used during tests.

Variable	Type
equipment_id	UUID
laboratory_id	UUID
equipment_code	VARCHAR
equipment_type	VARCHAR
manufacturer	VARCHAR
model	VARCHAR
serial_number	VARCHAR
accuracy	DECIMAL
accuracy_class	VARCHAR
range_min	DECIMAL
range_max	DECIMAL
unit	VARCHAR
calibration_certificate_no	VARCHAR

Variable	Type
calibration_date	DATE
calibration_due_date	DATE
status	ENUM
remarks	TEXT

16. test_equipment_usage

Links equipment to individual tests.

Variable	Type
usage_id	UUID
session_test_id	UUID
equipment_id	UUID
usage_purpose	TEXT
used_from	TIMESTAMP
used_until	TIMESTAMP

17. reports

Stores generated test reports.

Variable	Type
report_id	UUID
test_session_id	UUID
report_number	VARCHAR
report_version	INTEGER
report_status	ENUM
overall_result	ENUM

Variable	Type
generated_by	UUID
generated_at	TIMESTAMP
approved_by	UUID
approved_at	TIMESTAMP
docx_path	TEXT
pdf_path	TEXT
report_hash	VARCHAR
remarks	TEXT

18. report_versions

Stores historical versions of generated reports.

Variable	Type
report_version_id	UUID
report_id	UUID
version_number	INTEGER
generated_by	UUID
generated_at	TIMESTAMP
reason	TEXT
docx_path	TEXT
pdf_path	TEXT
hash	VARCHAR

19. attachments

Stores files associated with laboratories/tests/reports.

Variable	Type
attachment_id	UUID
laboratory_id	UUID
test_session_id	UUID
session_test_id	UUID nullable
file_name	VARCHAR
file_type	VARCHAR
file_size	BIGINT
file_path	TEXT
attachment_type	ENUM
uploaded_by	UUID
uploaded_at	TIMESTAMP
description	TEXT

20. audit_logs

Tracks who changed what and when.

Variable	Type
audit_id	UUID
laboratory_id	UUID
user_id	UUID
entity_type	VARCHAR
entity_id	UUID
action	ENUM
old_value	JSON
new_value	JSON
timestamp	TIMESTAMP

Variable	Type
ip_address	VARCHAR
remarks	TEXT

21. digital_signatures

Stores report approval/signature information.

Variable	Type
signature_id	UUID
report_id	UUID
user_id	UUID
signature_type	VARCHAR
signature_data	TEXT
signed_at	TIMESTAMP
certificate_id	VARCHAR
status	ENUM

Summary

#	Table
1	laboratories
2	users
3	instruments
4	standards
5	test_definitions
6	test_applicability_rules
7	mpe_rules

#	Table
8	test_sessions
9	test_session_tests
10	test_observations
11	observation_imports
12	test_calculations
13	test_results
14	environmental_conditions
15	test_equipment
16	test_equipment_usage
17	reports
18	report_versions
19	attachments
20	audit_logs
21	digital_signatures

21 tables total.

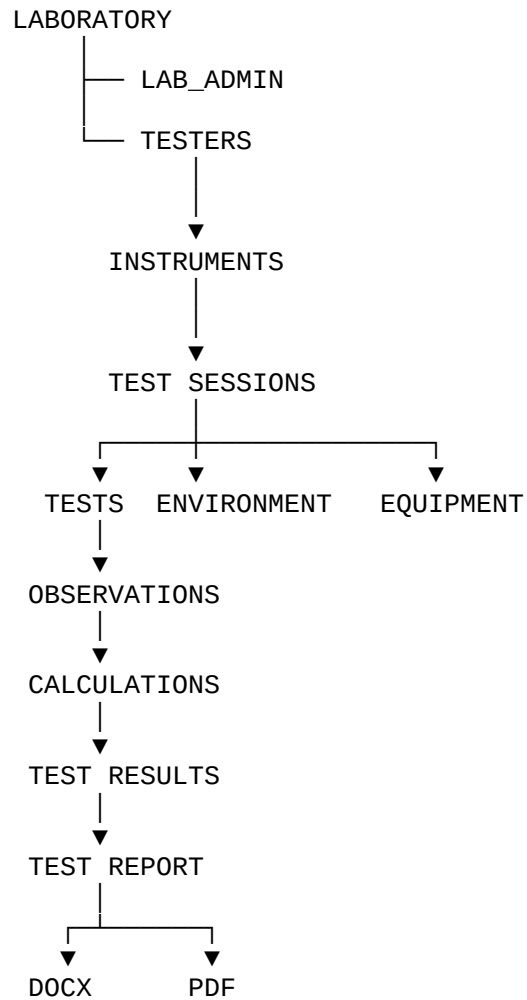
The original also mentions **calculation rules** conceptually, but it does **not define a separate calculation_rules table**; calculations are represented through test_definitions, test_calculations, and mpe_rules.

Detailed Schema

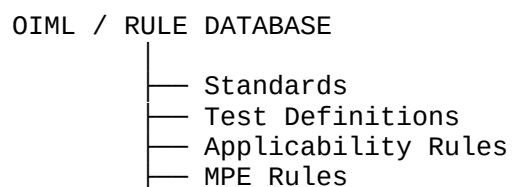
Yes. Before architecture, I would make **one canonical data model** that everyone on the team works from. The key is to keep **master data**, **test-session data**, **observations**, **calculated results**, and **generated reports** separate.

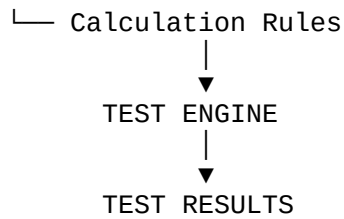
NAWI Test Report Website - Overall Data Schema

1. Core entity relationship



And separately:





2. laboratories

Stores registered testing laboratories.

Field	Type	Description
laboratory_id	UUID	Unique laboratory ID
laboratory_code	VARCHAR	Short lab identifier
name	VARCHAR	Official laboratory name
registration_number	VARCHAR	Laboratory registration/accreditation number
address_line_1	VARCHAR	Address
address_line_2	VARCHAR	Address
city	VARCHAR	City
state	VARCHAR	State
pincode	VARCHAR	PIN
country	VARCHAR	Country
phone	VARCHAR	Contact number
email	VARCHAR	Official email
website	VARCHAR	Optional
accreditation_body	VARCHAR	Accreditation authority
accreditation_number	VARCHAR	Accreditation number
accreditation_valid_from	DATE	Validity
accreditation_valid_until	DATE	Validity
status	ENUM	active, suspended, inactive
created_at	TIMESTAMP	Creation
updated_at	TIMESTAMP	Last update

3. users

Do not create separate login systems for admins and testers.

Have one user table with roles.

Field	Type	Description
user_id	UUID	Unique user
laboratory_id	UUID FK	Associated laboratory
employee_id	VARCHAR	Employee/staff ID
first_name	VARCHAR	First name
last_name	VARCHAR	Last name
email	VARCHAR	Login email
phone	VARCHAR	Contact
password_hash	VARCHAR	Hashed password
role	ENUM	lab_admin, tester, reviewer, approver
designation	VARCHAR	Job designation
qualification	TEXT	Qualification
authorization_scope	JSON	Tests/types they are authorized for
status	ENUM	active, inactive, suspended
last_login_at	TIMESTAMP	Last login
created_by	UUID FK	Who created account
created_at	TIMESTAMP	Creation
updated_at	TIMESTAMP	Update

Why this matters

Your flow becomes:

```

Laboratory registers
    ↓
Lab Admin account
    ↓
Admin adds tester
    ↓
System generates credentials
    ↓
Tester logs in

```

The **lab admin should never see another laboratory's data.**

4. instruments

This is your saved device/instrument master.

And **yes, I strongly recommend saving instruments.**

A laboratory may test the same model/type repeatedly. They shouldn't have to enter:

Max = 30 kg, Min = 200 g, Class III, e = 10 g...

every time.

Field	Type	Description
instrument_id	UUID	Unique instrument
laboratory_id	UUID FK	Owning laboratory
instrument_code	VARCHAR	Internal lab ID
manufacturer	VARCHAR	Manufacturer
model	VARCHAR	Model
type_designation	VARCHAR	Type designation
serial_number	VARCHAR	Serial number
instrument_type	ENUM/VARCHAR	Platform, bench, weighbridge, etc.
instrument_category	VARCHAR	NAWI category
accuracy_class	VARCHAR	I / II / III / IIII
Max	DECIMAL	Maximum capacity
Min	DECIMAL	Minimum capacity
e	DECIMAL	Verification scale interval
d	DECIMAL	Actual scale interval
n	INTEGER	Number of verification intervals
unit	VARCHAR	kg, g, t, etc.
tare_type	VARCHAR	Tare configuration
zero_setting_type	VARCHAR	Zero-setting mechanism
indication_type	VARCHAR	Digital / other
load_cell_manufacturer	VARCHAR	Load cell manufacturer
load_cell_model	VARCHAR	Load cell model
load_cell_capacity	DECIMAL	Capacity
software_version	VARCHAR	Instrument software
firmware_version	VARCHAR	Optional
power_supply	VARCHAR	Power details
interfaces	JSON	RS232, USB etc.
operating_temp_min	DECIMAL	°C
operating_temp_max	DECIMAL	°C
status	ENUM	active/inactive
created_by	UUID FK	Tester/admin
created_at	TIMESTAMP	Creation
updated_at	TIMESTAMP	Update

Important distinction

I'd actually support:

Instrument Model

Physical Instrument⁺

eventually.

For example:

```
ABC
├── EPS-30
│   ├── Serial 001
│   ├── Serial 002
│   └── Serial 003
```

But **for the MVP**, instruments can represent the actual submitted instrument.

5. standards

Stores the standards/rule versions used by the system.

Field	Type
standard_id	UUID
standard_code	VARCHAR
title	VARCHAR
version	VARCHAR
edition_year	INTEGER
effective_from	DATE
effective_until	DATE
source_document	VARCHAR
source_url	TEXT
status	ENUM
created_at	TIMESTAMP

Example:

```
OIML R 76-1
OIML R 76-2
```

This becomes important when the standard gets revised.

6. test_definitions

This is one of the **most important tables**.

It defines what a test actually is.

Field	Type	Description
test_definition_id	UUID	Test ID
standard_id	UUID FK	Applicable standard
test_code	VARCHAR	e.g. WP-001
test_name	VARCHAR	Weighing Performance
test_category	VARCHAR	Metrological / influence / disturbance etc.
description	TEXT	Test purpose
reference_clause	VARCHAR	OIML clause
procedure	TEXT/JSON	Test procedure
required_inputs	JSON	Required observations
calculation_type	VARCHAR	Calculation engine identifier
acceptance_rule	JSON	Acceptance condition
required_equipment	JSON	Equipment required
is_mandatory	BOOLEAN	Whether mandatory
active	BOOLEAN	Active definition

Example:

```
{
  "test_code": "WP-001",
  "test_name": "Weighing Performance",
  "reference_clause": "...",
  "calculation_type": "weighing_error",
  "required_inputs": [
    "load",
    "indication",
    "additional_load"
  ]
}
```

7. test_applicability_rules

This handles your **N/A requirement**.

Don't simply hide tests that don't apply.

The report should be able to say:

Eccentricity — N/A

and preferably explain why.

Field	Type
applicability_rule_id	UUID
test_definition_id	UUID FK
instrument_condition	JSON
applicable	BOOLEAN
reason	TEXT
priority	INTEGER
standard_clause	VARCHAR

Example:

```
{
  "instrument_type": "X",
  "condition": "specific construction",
  "applicable": false,
  "reason": "Test not applicable to this instrument configuration."
}
```

So your test list becomes:

Weighing Performance	✓ Applicable
Repeatability	✓ Applicable
Eccentricity	✓ Applicable
Discrimination	✓ Applicable
Tare	✓ Applicable
Creep	✓ Applicable
Some Test	N/A
Another Test	N/A

And N/A tests remain visible in the final report.

8. mpe_rules

Stores permissible-error rules.

Field	Type
mpe_rule_id	UUID
standard_id	UUID FK
accuracy_class	VARCHAR
range_min_e	DECIMAL
range_max_e	DECIMAL
mpe_value_e	DECIMAL
mpe_unit_type	VARCHAR
condition	VARCHAR

Field	Type
test_type	VARCHAR
reference_clause	VARCHAR

Example:

Class III

0 - 500e	→ ±0.5e
>500e - 2000e	→ ±1e
>2000e - 10000e	→ ±1.5e

The calculation engine uses this rather than hardcoding numbers throughout the application.

9. test_sessions

This is the **actual testing event**.

Think:

"Tester X tested Instrument Y on Date Z using Standard Version V."

Field	Type
test_session_id	UUID
laboratory_id	UUID FK
instrument_id	UUID FK
tester_id	UUID FK
reviewer_id	UUID FK
standard_id	UUID FK
session_number	VARCHAR
application_number	VARCHAR
test_type	VARCHAR
started_at	TIMESTAMP
completed_at	TIMESTAMP
status	ENUM
overall_result	ENUM
remarks	TEXT
created_at	TIMESTAMP
updated_at	TIMESTAMP

Status:

draft
in_progress
submitted
under_review
approved
rejected
completed
cancelled

10. test_session_tests

This connects a test session to **every applicable/N-A test**.

This is important.

Field	Type
session_test_id	UUID
test_session_id	UUID FK
test_definition_id	UUID FK
applicability_status	ENUM
na_reason	TEXT
status	ENUM
result	ENUM
started_at	TIMESTAMP
completed_at	TIMESTAMP

Example:

TEST SESSION #0017

Weighing Performance	Applicable	PASS
Repeatability	Applicable	PASS
Eccentricity	Applicable	PASS
Tare	Applicable	PASS
Creep	Applicable	PASS
Temperature	Applicable	PASS
Some Test	N/A	—

11. test_observations

This stores the **actual values entered by the tester**.

This should be separate from calculated results.

Field	Type
observation_id	UUID
session_test_id	UUID FK
parameter_name	VARCHAR
parameter_code	VARCHAR
value_numeric	DECIMAL
value_text	TEXT
unit	VARCHAR
timestamp	TIMESTAMP
sequence_no	INTEGER
source	ENUM
entered_by	UUID FK
remarks	TEXT

source:

manual
 excel_import
 csv_import
 instrument_import

This supports your idea perfectly.

12. Excel/CSV Import

I'd make a separate table:

observation_imports

Field	Type
import_id	UUID
test_session_id	UUID
uploaded_by	UUID
file_name	VARCHAR
file_type	VARCHAR
file_path	TEXT
row_count	INTEGER
valid_rows	INTEGER
invalid_rows	INTEGER
validation_status	ENUM
validation_errors	JSON

Field	Type
uploaded_at	TIMESTAMP

Workflow:

```

Excel
  ↓
Upload
  ↓
Parse
  ↓
Validate
  ↓
Preview
  ↓
Confirm
  ↓
test_observations

```

Don't directly import Excel into the database without validation.

13. test_calculations

Stores values **calculated by the system**.

This is different from observations.

Field	Type
calculation_id	UUID
session_test_id	UUID
observation_id	UUID
calculation_type	VARCHAR
input_values	JSON
formula	TEXT
calculated_value	DECIMAL
unit	VARCHAR
calculation_version	VARCHAR
calculated_at	TIMESTAMP

Example:

Input:
 Load = 10.00 kg
 Indication = 10.01 kg

Formula:
Error = Indication - Load

Result:
Error = +0.01 kg

14. test_results

This stores the **final interpretation**.

Field	Type
result_id	UUID
session_test_id	UUID
measured_value	DECIMAL
mpe_value	DECIMAL
error_value	DECIMAL
corrected_error	DECIMAL
acceptance_condition	TEXT
pass_fail	ENUM
result_summary	TEXT
calculation_version	VARCHAR
evaluated_at	TIMESTAMP

Possible result:

PASS
FAIL
N/A
INCONCLUSIVE

15. environmental_conditions

Stores lab conditions during testing.

Field	Type
environment_id	UUID
test_session_id	UUID
temperature	DECIMAL
temperature_unit	VARCHAR
humidity	DECIMAL
humidity_unit	VARCHAR
pressure	DECIMAL

Field	Type
pressure_unit	VARCHAR
recorded_at	TIMESTAMP
recorded_by	UUID
source	VARCHAR
remarks	TEXT

For tests requiring time-series environmental data, multiple records can be stored.

16. test_equipment

Stores equipment used by the laboratory.

Field	Type
equipment_id	UUID
laboratory_id	UUID
equipment_code	VARCHAR
equipment_type	VARCHAR
manufacturer	VARCHAR
model	VARCHAR
serial_number	VARCHAR
accuracy	DECIMAL
accuracy_class	VARCHAR
range_min	DECIMAL
range_max	DECIMAL
unit	VARCHAR
calibration_certificate_no	VARCHAR
calibration_date	DATE
calibration_due_date	DATE
status	ENUM
remarks	TEXT

17. test_equipment_usage

Because one piece of equipment can be used in many tests.

Field	Type
usage_id	UUID

Field	Type
session_test_id	UUID
equipment_id	UUID
usage_purpose	TEXT
used_from	TIMESTAMP
used_until	TIMESTAMP

18. reports

This is your **final generated report record**.

Field	Type
report_id	UUID
test_session_id	UUID
report_number	VARCHAR
report_version	INTEGER
report_status	ENUM
overall_result	ENUM
generated_by	UUID
generated_at	TIMESTAMP
approved_by	UUID
approved_at	TIMESTAMP
docx_path	TEXT
pdf_path	TEXT
report_hash	VARCHAR
remarks	TEXT

So:

Report #NAWI-TE-2026-00017

```

├─ report.docx
├─ report.pdf

```

19. report_versions

I'd strongly recommend this.

Because if someone modifies a report after generation, you don't want the previous version disappearing.

Field	Type
report_version_id	UUID
report_id	UUID
version_number	INTEGER
generated_by	UUID
generated_at	TIMESTAMP
reason	TEXT
docx_path	TEXT
pdf_path	TEXT
hash	VARCHAR

Example:

Report 00017

- └─ v1 → Generated
- └─ v2 → Corrected observation
- └─ v3 → Approved

20. attachments

For photographs, calibration certificates, Excel files, etc.

Field	Type
attachment_id	UUID
laboratory_id	UUID
test_session_id	UUID
session_test_id	UUID nullable
file_name	VARCHAR
file_type	VARCHAR
file_size	BIGINT
file_path	TEXT
attachment_type	ENUM
uploaded_by	UUID
uploaded_at	TIMESTAMP
description	TEXT

Types:

instrument_photo
test_setup_photo
nameplate_photo
calibration_certificate
excel_input
raw_data

21. audit_logs

This is particularly important for a testing/compliance application.

You want to know:

Who changed what, and when?

Field	Type
audit_id	UUID
laboratory_id	UUID
user_id	UUID
entity_type	VARCHAR
entity_id	UUID
action	ENUM
old_value	JSON
new_value	JSON
timestamp	TIMESTAMP
ip_address	VARCHAR
remarks	TEXT

Actions:

CREATE
UPDATE
DELETE
IMPORT
SUBMIT
APPROVE
REJECT
GENERATE_REPORT
DOWNLOAD_REPORT
LOGIN

22. digital_signatures

For your optional signature functionality.

Field	Type
signature_id	UUID

Field	Type
report_id	UUID
user_id	UUID
signature_type	VARCHAR
signature_data	TEXT
signed_at	TIMESTAMP
certificate_id	VARCHAR
status	ENUM

For MVP, this can simply be a **digital approval/signature placeholder** rather than a legally binding digital-signature implementation.

23. Dashboard / analytics

You **don't need a separate dashboard database** initially.

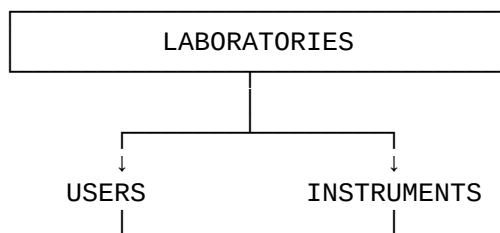
Calculate from existing records:

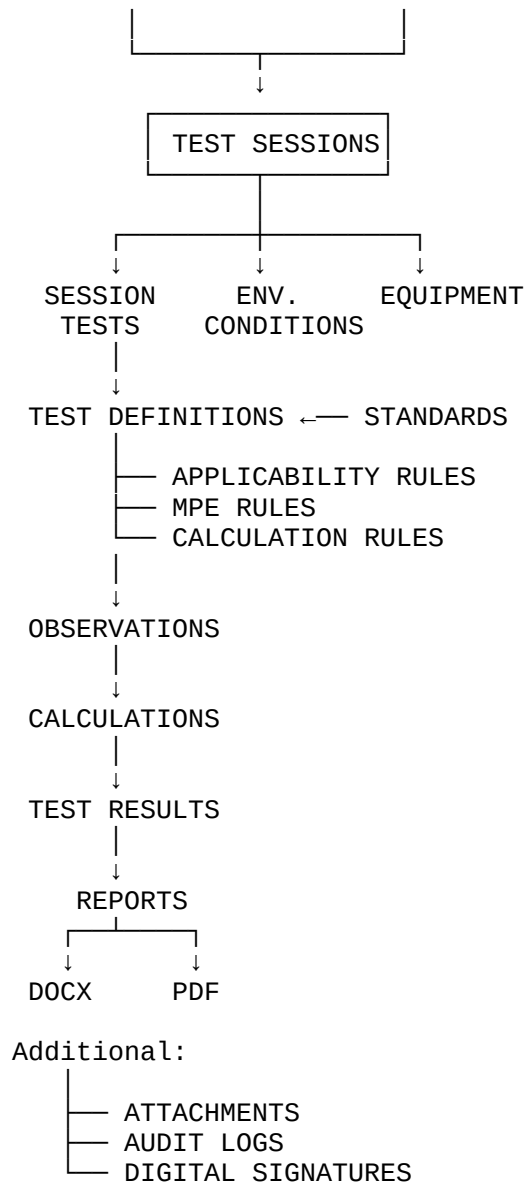
Total tests
 Tests completed
 Tests pending
 Reports generated
 Reports awaiting approval
 PASS
 FAIL
 N/A
 Tests by month
 Tests by laboratory

If performance becomes an issue, add aggregated tables later.

24. The complete schema at a glance

This is the part I'd actually give to the team:





25. The most important conceptual separation

Tell everyone on the team this:

A. Master Data

Things that exist independently of a test:

Laboratory
Users

Instruments
Test Equipment
Standards
Test Definitions
MPE Rules
Applicability Rules

B. Test Data

Things created when a test is performed:

Test Session
Selected Tests
Environmental Conditions
Observations
Calculations
Results

C. Output / Evidence

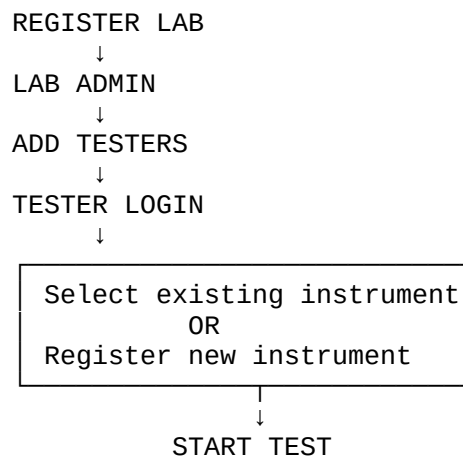
Things generated from the test:

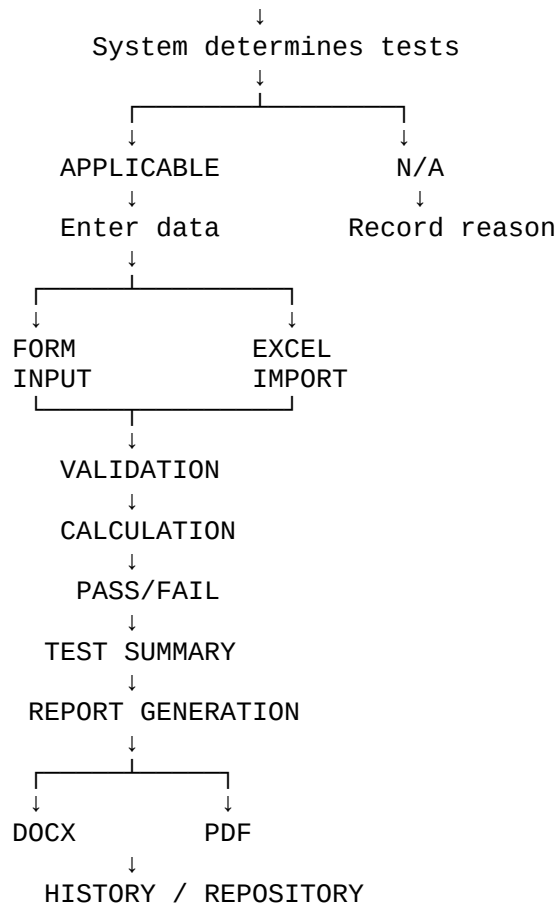
Report
Report Versions
Attachments
Digital Signatures
Audit Trail

That separation will prevent a **lot** of confusion when six people start developing simultaneously.

26. Your actual workflow using this schema

The website then becomes:





And **yes, saving previously entered instruments is the right design**. The tester can have:

My Instruments → Select Instrument → Start New Test

while still having the option:

+ Register New Instrument

That will make repeated testing substantially easier.

One thing *not* finalized yet

I would **not yet lock the exact fields inside test_observations, test_calculations, test_results, and test_definitions**.

Those need to be derived **test-by-test from OIML R 76-1/R 76-2**, because a weighing-performance test needs different observations from, say, eccentricity, creep, temperature, or electrical-disturbance testing.

So this gives your team the **overall website/database contract**, while the next step should be to create the **detailed schema for every individual OIML test**. That is where we'll decide exactly what the tester sees in each form, what an Excel template looks like, what gets calculated, and what goes into each report section.

For actual data for tests (yeh Jatin kar rha hai):

The next step is simply:

1. Take each OIML R76 test.
2. For each test, define:
 - o **Inputs** — what tester enters
 - o **Observations** — what is measured
 - o **Calculations** — what software computes
 - o **Requirement/MPE** — what it is compared against
 - o **Result** — PASS / FAIL / N/A
3. Turn those into:
 - o database fields
 - o tester form fields
 - o Excel template columns
 - o report fields

For example:

Weighing Performance

Inputs: Load, Indication
Calculate: Error
Requirement: MPE
Result: PASS/FAIL

Repeatability

Inputs: Test load, multiple indications
Calculate: Maximum difference
Requirement: Applicable limit
Result: PASS/FAIL

Eccentricity

Inputs: Position, Load, Indication
Calculate: Error at each position
Requirement: MPE
Result: PASS/FAIL

So yes: our next task should be to go test-by-test through R76 and build this exact structure.